

```

void CJanuar2023View::DrawArcSpot(CDC* pDC, int size, COLORREF colFill, int widthLine,
COLORREF colLine)
{
    CPen pen(PS_SOLID, widthLine, colLine);

    CBrush brush(colFill);

    CPen* oldPen = pDC->SelectObject(&pen);
    CBrush* oldBrush = pDC->SelectObject(&brush);

    // komentar: crtamo tri spojena bezier segmenta (PolyBezier)
    // - levi unutrašnji: (0,r) -> (r,0)
    // - srednji iznad: (r,0) -> (3r,0)
    // - desni unutrašnji: (3r,0) -> (4r,r)

    int r = size / 4;

    const double k = 0.5522847498; // standardna konstanta

    CPoint pts[19];

    // Levi unutrašnji polukrug
    // start (0,r)
    pts[0] = CPoint(0, r);

    // kontrolne tacke koje guraju krivu *unutra* (y veka, x veka)
    pts[1] = CPoint(int(k * r), r);
    pts[2] = CPoint(r, int(k * r));
    pts[3] = CPoint(r, 0); // kraj prvog segmenta

```

```
// Srednji luk iznad gornje stranice  
// start implicitno = pts[3] == (r,0)  
pts[4] = CPoint(int(r + k * r), int(- r - k * r)); // CP1 iznad (negativni y)  
pts[5] = CPoint(int(3 * r - k * r), int(- r - k * r)); // CP2 iznad  
pts[6] = CPoint(3 * r, 0); // kraj drugog segmenta
```

```
// Desni unutrašnji polukrug (simetričan prvom)  
pts[7] = CPoint(3 * r, int(k * r));  
pts[8] = CPoint(int(4 * r - k * r), r);  
pts[9] = CPoint(4 * r, r); // = (size, r)
```

```
//Srednji desni  
pts[10] = CPoint(int(5 * r + k * r), int(r + k * r));  
pts[11] = CPoint(int(5 * r + k * r), int(3 * r - k * r));  
pts[12] = CPoint(4 * r, 3 * r);
```

```
// Donji desni unutrašnji luk  
pts[13] = CPoint(int(4 * r - k * r), int(3 * r + k * r));  
pts[14] = CPoint(int(3 * r + k * r), int(4 * r - k * r));  
pts[15] = CPoint(int(3 * r + k * r), 4 * r);
```

```
//Donji srednji luk  
//pts[16] = CPoint(int(3 * r - 2 * k * r), (5 * r + k * r));  
//pts[17] = CPoint(int(r + k * r), int(5 * r + k * r));  
//pts[18] = CPoint(r, 4 * r);  
// Donji srednji luk ☐ konkavan ka spolja ispod kvadrata
```

```

// start = (3r, 4r)

pts[16] = CPoint(int(3 * r - k * r), int(5 * r)); // cp1, ide udesno-nadole
pts[17] = CPoint(int(r + k * r), int(5 * r)); // cp2, ide ulevo-nadole
pts[18] = CPoint(r, 4 * r); // end, ulazi u levi donji ugao kvadrata


// crtanje
pDC->BeginPath();
pDC->MoveTo(pts[0]); // pocetak
pDC->PolyBezier(pts, 19); // crta 3 kote (svaki segment 3 kontrolne + kraj)
pDC->EndPath();
pDC->StrokePath();

pDC->SelectObject(oldPen);
pDC->SelectObject(oldBrush);
}

```

```

void CJanuar2024View::DrawPetal(CDC* pDC, CRect rect, COLORREF clrFill, COLORREF clrLine)
{
    CPen* olovkaLinija = new CPen(PS_SOLID, 1, clrLine);
    CBrush* cetkicalspune = new CBrush(clrFill);
    CPen* staraOlovka = pDC->SelectObject(olovkaLinija);
    CBrush* staraCetkica = pDC->SelectObject(cetkicalspune);

    int startX = rect.left;
    int startY = rect.bottom;

```

```
pDC->MoveTo(startX, startY);  
pDC->BeginPath();  
pDC->LineTo(CPoint(startX, startY - (rect.Height()/20)));  
pDC->LineTo(CPoint(startY - (rect.Height()/20), rect.top));  
pDC->LineTo(CPoint(startX + (rect.Width() / 20), startY));  
pDC->LineTo(CPoint(startX, startY));  
pDC->EndPath();  
pDC->StrokeAndFillPath();
```

```
pDC->MoveTo(rect.right - rect.Width() / 2, rect.bottom - rect.Height() / 2);  
pDC->BeginPath();  
pDC->LineTo(rect.right - rect.Width() / 2 - 2, rect.top);  
pDC->LineTo(rect.right - rect.Width() / 2 + (rect.Width() / 20) + 2, rect.top);  
pDC->LineTo(rect.right - rect.Width() / 2, rect.bottom - rect.Height() / 2);  
pDC->EndPath();  
pDC->StrokeAndFillPath();
```

```
pDC->MoveTo(rect.right - rect.Width() / 2, rect.bottom - rect.Height() / 2 - 5);  
pDC->BeginPath();  
pDC->LineTo(rect.right, rect.bottom - rect.Height() / 2 - 7);  
pDC->LineTo(rect.right, rect.bottom - rect.Height() / 2 - (rect.Height() / 20) + 7);  
pDC->LineTo(rect.right - rect.Width() / 2, rect.bottom - rect.Height() / 2 - 5);  
pDC->EndPath();  
pDC->StrokeAndFillPath();
```

```
pDC->SelectObject(staraOlovka);
```

```
pDC->SelectObject(staraCetkica);  
delete olovkaLinija;  
delete cetkicalspune;  
}
```

```
void CJanuar2024View::DrawSnowFlake(CDC* pDC, CRect rect, COLORREF clrFill, COLORREF  
clrLine, int nArcs, float dRot, CString str)
```

```
{  
    int prevGraphicsMode = SetGraphicsMode(pDC->m_hDC, GM_ADVANCED);  
    XFORM transOld;  
    pDC->GetWorldTransform(&transOld);  
    CRect rect2;  
    GetClientRect(&rect2);  
    //Crtanje n delova pahuljice  
    double korak = 2 * PI / nArcs;  
    for (int i = 0; i < nArcs; i++)  
    {  
        // Resetuj transformaciju na originalnu  
        pDC->SetWorldTransform(&transOld);  
        // Translacija na centar ekrana  
        //Translate(pDC, (float)rect2.Width() / 2, (float)rect2.Height() / 2, false);  
        Rotate(pDC, dRot, false);  
        // Rotacija za ugao ove latice  
        Rotate(pDC, i * korak, false);  
  
        // Translacija gore da donji levi ugao latice bude u centru
```

```
Translate(pDC, 0, -rect.Height(), false);
```

```
// Nacrtaj latice
```

```
DrawPetal(pDC, rect, clrFill, clrLine);
```

```
}
```

```
// Centar pahuljice
```

```
pDC->SetWorldTransform(&transOld);
```

```
CBrush* centarPahuljiceCetka = new CBrush(HS_DIAGCROSS, RGB(0, 0, 255));
```

```
CBrush* staraCetka = pDC->SelectObject(centarPahuljiceCetka);
```

```
int radius = 20;
```

```
CRect rectCircle(-radius, -radius, radius, radius);
```

```
pDC->Ellipse(&rectCircle);
```

```
//Teskt
```

```
CFont font;
```

```
font.CreateFont(20, 0, 0, 0, FW_BOLD, 0, 0, 0, RUSSIAN_CHARSET, 0, 0, 0, 0, _T("Arial"));
```

```
CFont* stariFont = pDC->SelectObject(&font);
```

```
int stariBkMode = pDC->SetBkMode(TRANSPARENT); // Ovo je neophodno da mi centri  
pahuljica ne bi bili transparentni
```

```
pDC->SetTextColor(RGB(0, 0, 0));
```

```
int n = str.GetLength();
```

```
double r = 1.5 * rect.Height();
```

```
double pocetniUgao = 135.0 * PI / 180.0; //u radijanima je ugao, 135 stepeni je pocetni, pise u  
tekstu zadatka
```

```
double krajnjiUgao = 45.0 * PI / 180.0; //45 stepeni je krajnji ugao
```

double korakTekst = (krajnjiUgao - pocetniUgao) / (n - 1); // n - 1 karaktera u stringu, korak se dobija tako sto se krajnji ugao oduzme

double novoX = 0, novoY = 0;

// od pocetnog do krajnjeg karaktera

for (int i = 0; i < n; i++)

{

double ugao = pocetniUgao + i * korakTekst;

pDC->SetWorldTransform(&transOld);

// Translacija na centar pahuljice

Translate(pDC, 0, 0, false);

novoX = (double)(r * cos(ugao)); // Novo x na koje se pomeram

novoY = (double)(r * sin(ugao)); // Novo y na koje se pomeram

// Translacija na poziciju slova, ovo novo x i y su zapravo nove tacke koje dobijam i koje mi predstavljaju centar slova

Translate(pDC, novoX, novoY, false);

// Rotacija tako da slovo stoji tangentno

Rotate(pDC, ugao - PI / 2, false);

// Ispis slova (slovo se crta u centru)

CString s;

s.Format(L"%c", str[i]);

pDC->TextOut(0, 0, s);

```
}
```

```
pDC->SetBkMode(stariBkMode); // Vracam stari BkMode
```

```
pDC->SelectObject(staraCetka);
```

```
pDC->SelectObject(stariFont);
```

```
pDC->SetWorldTransform(&transOld);
```

```
pDC->SetGraphicsMode(prevGraphicsMode);
```

```
delete centarPahuljiceCetka;
```

```
}
```

```
void CJanuar2024View::DrawSnowFlakes(CDC* pDC, CRect* aRect, COLORREF* aClrFill,  
COLORREF* sClrLine, int* anArcs, float* aRot, CString* aStr, CPoint* aPts)
```

```
{
```

```
int brojPahuljica = 0;
```

```
while (!(aPts[brojPahuljica].x == -1 && aPts[brojPahuljica].y == -1))
```

```
    brojPahuljica++;
```

```
for (int i = 0; i < brojPahuljica; i++)
```

```
{
```

```
    int prevGraphicsMode = SetGraphicsMode(pDC->m_hDC, GM_ADVANCED);
```

```
    XFORM transOld;
```

```
    pDC->GetWorldTransform(&transOld);
```

```
    Translate(pDC, aPts[i].x, aPts[i].y, false);
```

```
    DrawSnowFlake(pDC, *aRect, aClrFill[i], sClrLine[i], anArcs[i], aRot[i], aStr[i]);
```

```
    pDC->SetWorldTransform(&transOld);
```



```

        pDC->SetGraphicsMode(prevGraphicsMode);
    }
}

void CJanuar2024View::OnDraw(CDC* pDC)
{
    CJanuar2024Doc* pDoc = GetDocument();
    ASSERT_VALID(pDoc);
    if (!pDoc)
        return;

    // TODO: add draw code for native data here

    CRect rect;
    GetClientRect(&rect);

    CString tekstovi[3] = { L"***GDI***", L"Grafika", L"Januar 2024" }; // Ovde treba cirilica, ali ja
    iskreno ne znam kako da to podesim u visual studio

    CPoint centri[4] = { CPoint(200, 200), CPoint(rect.Width()/2, 450), CPoint(1200, 200), CPoint(-
    1, -1)};

    CRect rectPahuljice(0, 0, 100, 100);

    COLORREF clrFills[3] = { RGB(0, 0, 255), RGB(255, 0, 0), RGB(0, 255, 0) };
    COLORREF clrLines[3] = { RGB(0, 0, 255), RGB(255, 0, 0), RGB(0, 255, 0) };
    int nArcs[3] = { 3, 6, 9 };
    float dRots[3] = { 0.0f, 45.0f, 75.0f };

    DrawSnowFlakes(pDC, &rectPahuljice, clrFills, clrLines, nArcs, dRots, tekstovi, centri);
}

```

```
void CSeptembar2023View::DrawFilmReel(CDC* pDC, int r, int n, int dBig, int rBig, int dSmall, int rSmall, COLORREF colFill, int wLine, COLORREF colLine)
```

```
{
    CPen* olovkalvicaSvihKontura = new CPen(PS_SOLID, wLine, colLine);
    CBrush* cetkicalspuneKontura = new CBrush(colFill);
    CBrush* belaCetkica = new CBrush(RGB(255, 255, 255));
    CPen* staraOlovka = pDC->SelectObject(olovkalvicaSvihKontura);
    CBrush* staraCetkica = pDC->SelectObject(cetkicalspuneKontura);
    pDC->Ellipse(-r, -r, +r, +r);
    pDC->SelectObject(belaCetkica);
    //n vecih kruznica
    double novoX = 0;
    double novoY = 0;
    double korak = 2 * PI / n;
    for (double alfa = PI / 2; alfa < 2 * PI + PI / 2; alfa += korak)
    {
        novoX = dBig * cos(alfa);
        novoY = dBig * sin(alfa);
        pDC->Ellipse(novoX-rBig, novoY-rBig, novoX+rBig, novoY+rBig);
    }
    //4 manje kruznice
    novoX = 0;
    novoY = 0;
    korak = 2 * PI / 4;
    for (double alfa = PI / 2; alfa < 2 * PI + PI / 2; alfa += korak)
    {
```

```

    novoX = dSmall * cos(alfa);
    novoY = dSmall * sin(alfa);
    pDC->Ellipse(novoX - rSmall, novoY - rSmall, novoX + rSmall, novoY + rSmall);
}
pDC->SelectObject(staraOlovka);
pDC->SelectObject(staraCetkica);
delete cetskicalspuneKontura;
delete belaCetkica;
delete olovkalvicaSvihKontura;
}

```

```

void CSeptembar2023View::DrawFilmReel(CDC* pDC, int r, int n, int dBig, int rBig, int dSmall, int
rSmall, COLORREF colFill, int wLine, COLORREF colLine, double angle, int nText, CString strText[],
int spacing[], int hFont, COLORREF colText)

```

```

{
    int prevMode = SetGraphicsMode(pDC->m_hDC, GM_ADVANCED);
    XFORM xFormOld;
    pDC->GetWorldTransform(&xFormOld);
    Translate(pDC, 3.5*r, 1.5*r, false);
    Rotate(pDC, angle, false);
    DrawFilmReel(pDC, r, n, dBig, rBig, dSmall, rSmall, colFill, wLine, colLine);
    //Fontovi
    CFont font;
    //Moram da se vratim nazad neke transformacije
    Rotate(pDC, -angle, false); // vracam ugao da bih tekst ispisao pravo, a ne pod angle uglom,
jer sam gore izvrsio rotaciju za ugao angle kod kotura
}

```

```
font.CreateFont(hFont, 0, 0, 0, FW_BOLD, 100, 0, 0, 0, 0, 0, 0, _T("Tahoma")); //kreiram font  
prema tekstu zadatka
```

```
CFont* stariFont = pDC->SelectObject(&font);
```

```
pDC->SetTextColor(colText);
```

pDC->SetBkMode(TRANSPARENT); //Ovo moze i ne mora, cisto da tekst nema svoju pozadinu (pravougaonik)

```
pDC->SetTextAlign(TA_RIGHT); //ravnam u desno
```

```
double baseX = -1.2*r; // desna ivica teksta je na -1.2*r udaljenosti, levo od centra kotura u
koji sam vec pozicioniran
```

```
int currentY = -r; // ravnjam se sa gornjom ivicom kotura, ja sam trenutno u centru kotura,
samo se vratim za poluprecnik do gornje ivice
```

```
for (int i = 0; i < nText; ++i)
```

 $\{$

```
pDC->TextOut((int)baseX, currentY, strText[i]);
```

```
currentY += spacing[i];
```

}

// Filmska traka (bezier kriva), nije bas najsretnije

```
CPen* bezierOlovka = new CPen(PS_SOLID, wLine, colLine);
```

```
CPen* staraOlovka = pDC->SelectObject(bezierOlovka);
```

```
//Moram bre da vratim transformacije bre, majmune
```

```
Translate(pDC, -3.5 * r, r - 0.25 * r, false);
```

```
Rotate(pDC, -angle, false);
```

```
double yKoordinataNeparnihTacaka = r + 0.25 * r;
```

```
//Za tacku jedan i dva koristim formulu kruznice za kruznicu sa centrom u tackama x0,y0: (x-
x0)^2+(y-y0)^2=r^2, x i y su tacke koje trazim, u ovom slucaju samo x
```

```
double xKoordinataPrveTacke = 245; //uradjena jednacina
```

```
double xKoordinataDrugeTacke = xKoordinataPrveTacke - 0.25 * r;
```

```

    double xKoordinataTreceTacke = xKoordinataDrugeTacke - ((xKoordinataPrveTacke -
xKoordinataDrugeTacke) * 0.8);

    double xKoordinataCetvrteTacke = xKoordinataTreceTacke - ((xKoordinataDrugeTacke -
xKoordinataTreceTacke) * 0.8);

    double xKoordinataPeteTacke = xKoordinataCetvrteTacke - ((xKoordinataTreceTacke -
xKoordinataCetvrteTacke) * 0.8);

    double xKoordinataSesteTacke = xKoordinataPeteTacke - ((xKoordinataCetvrteTacke -
xKoordinataPeteTacke) * 0.8);

    double xKoordinataSedmeTacke = xKoordinataSesteTacke - ((xKoordinataPeteTacke -
xKoordinataSesteTacke) * 0.8);

    POINT tacke[7] = { CPoint(xKoordinataSedmeTacke , yKoordinataNeparnihTacaka),
CPoint(xKoordinataSesteTacke ,yKoordinataNeparnihTacaka - 0.15 * r),
CPoint(xKoordinataPeteTacke ,yKoordinataNeparnihTacaka), CPoint(xKoordinataCetvrteTacke ,
yKoordinataNeparnihTacaka + 0.10 * r), CPoint(xKoordinataTreceTacke
,yKoordinataNeparnihTacaka), CPoint(xKoordinataDrugeTacke ,yKoordinataNeparnihTacaka -
0.05 * r), CPoint(xKoordinataPrveTacke ,yKoordinataNeparnihTacaka) };

    pDC->PolyBezier(tacke, 7);

    // Vрати stare objekte

    pDC->SelectObject(staraOlovka);

    pDC->SelectObject(&stariFont);

    pDC->SetWorldTransform(&xFormOld); // vraća koordinatni pocetak gde je bio pre
transformacije

    pDC->SetGraphicsMode(prevMode);

    delete bezierOlovka;

}

void CSeptembar2023View::Save(CDC* pDC, CRect rcDC, CRect rcBmp, CString sFile)
{

```

```

    //Ovo pojma nemam
}

void CSeptembar2023View::OnDraw(CDC* pDC)
{
    CSeptembar2023Doc* pDoc = GetDocument();
    ASSERT_VALID(pDoc);
    if (!pDoc)
        return;

    // TODO: add draw code for native data here

    //Tekst izdvojen na reci
    CString lines[3];
    lines[0] = _T("FILMSKI");
    lines[1] = _T("SUSRET");
    lines[2] = _T("NIS");

    int spacing[2] = { 40, 170 }; // razmaci izmedju redova, razmaci izmedju donje ivice prethodne,
    i gornje ivice sledece reci

    //40 je razmak izmedju donje ivice reci FILMSKI i gornje ivice reci SUSRET
    //170 je razmak izmedju donje ivice reci SUSRET, i gornje ivice reci NIS

    DrawFilmReel(pDC, 100, 6, 63, 25, 20, 10, RGB(125, 125, 125), 3, RGB(0, 0, 0), PI/6, 3, lines,
    spacing, 50, RGB(0, 0, 0));
}

```